SME 309 Microprocessor Design -- Final Project

2025 Fall

Project Requirements

1. Implementation of a five-stage pipeline ARM Processor [40 points]

In this project, you will implement a five-stage pipeline processor based on the single-cycle processor you developed in Lab2, as demonstrated in the lecture. The architecture is shown below. Take care of data hazards (data forwarding, stall, flush) and control hazards (early BTA, flush).

What's more, when a multi-cycle instruction is being executed, the CPU should continue executing subsequent instructions, provided there is no data dependency between the previous instruction and the current one. For example, instruction 1 is

MUL R5, R6, R7

And the next instruction (instruction 2) is

ADD R1, R2, R3

There is no data dependency between instruction 2 and instruction 1. When CPU is executing instruction 1, it can execute instruction 2 at the same time. Because Mcycle is an independent module, when it is busy, other parts of CPU can handle other instructions at the same time.

However, if instruction 2 is

ADD R1, R5, R3

The instruction 2 has a data dependency on instruction 1. In this case, the pipeline should stall until the result of the instruction 1 is available.

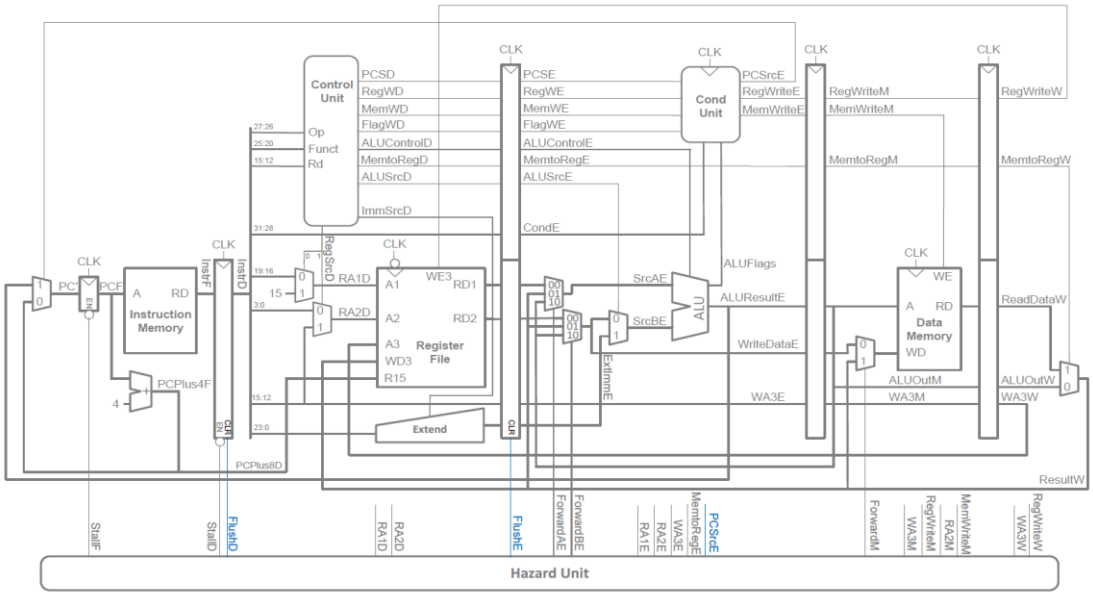
The following techniques should be implemented to handle pipeline hazards (please refer to the lecture notes for details):

(1) Using Different Clock Edges

(2) Inserting NOPs

(3) Data Forwarding

(4) Early BTA



Test requirements:

You are required to develop a testbench and corresponding assembly code to verify the correct operation of your five-stage pipeline processor in the simulation waveform. Ensure that the simulation waveforms demonstrate the following conditions, and include **annotated** screenshots in your report:

(1) Read-After-Write (RAW) hazard

(2) Memory-to-Memory copy

(3) Load-and-Use Hazard

(4) Control hazards

(5) No data dependency -- The pipeline continues execution without stalling.

(6) With data dependency -- The pipeline stalls appropriately until the result of the multi-cycle instruction is available.

Report requirements:

(1) Introduction (overview of the five-stage pipeline processor) **[5 points]**

(2) Hardware architecture (schematic and description of the pipeline stages) **[5 points]**

(3) Hazard handling techniques (data hazards and data forwarding mechanism, control hazards and early BTA, and other methods) **[8 points]**

(4) Implementation details (description of key modules) **[8 points]**

(5) Test code and simulation waveforms (testbench and assembly code for each hazard, and the waveform results) **[10 points]**

(6) On board test result **[4 points]**

2. Design and Benchmarking of a RISC-V Processor [60 points]

In this task, you will implement a CPU core to support simple RISC-V ISA. Your design should support basic instructions defined in the RV32I extension (Refer to RISC-V reference manual for more details), which is the minimal implementation for a RISC-V CPU.

Upon completion, we will provide a benchmark suite consisting of functional tests and a performance benchmarking program. You must first ensure your design passes the functional tests to verify correctness. Once verified, the benchmarking program will assess your CPU's performance; higher scores will directly contribute to a better grade.

To achieve superior performance, consider implementing advanced architectural features such as **Pipeline**, **Cache**, and **Dynamic Branch Prediction**. For further details, please refer to the **Appendix**.

Test Requirements:

Ensure that the CPU can correctly fetch, decode, execute, and write back results for all the required instruction types. Test your design by writing an assembly program yourself. Ensure that the simulation waveforms demonstrate the expected results, and include **annotated** screenshots in your report.

On Board Requirements:

While ensuring the correctness of instruction execution, adjust the clock period to maximize the benchmark score. The seven-segment display must show the correct data.

Report Requirements:

Your report should include the following:

(1) An overview of the RISC-V ISA, and comparison with the ARM architecture **[5 points]**

- Provide a concise overview of the RISC-V instruction set architecture.
- Compare RISC V with ARM, highlighting the key differences in hardware design, instruction encoding, and architectural philosophy.

(2) Description of CPU modules and implementation **[5 points]**

- Describe each hardware module of your RISC-V core and explain how they are implemented (e.g., IF, ID, EX, MEM, WB, register file, ALU, control unit, pipeline registers, multi cycle units, etc.).

(3) CPU simulation waveforms **[25 points]**

- Include **annotated** simulation waveforms that illustrate correct functional behavior, pipeline activity, hazard handling, and any special cases you tested.

(4) Benchmark result of single cycle CPU and the optimized CPU. **[20 points]**

- Present and compare benchmark scores (from the provided benchmark suite) for the baseline single cycle CPU and your optimized implementation. Include methodology and configuration details.

(5) Implementation challenges and reflections **[5 points]**

- Discuss the major challenges you encountered during the RISC-V core implementation and insights, trade-offs, or design inspirations you derived from the project.

Bonus: Git Usage

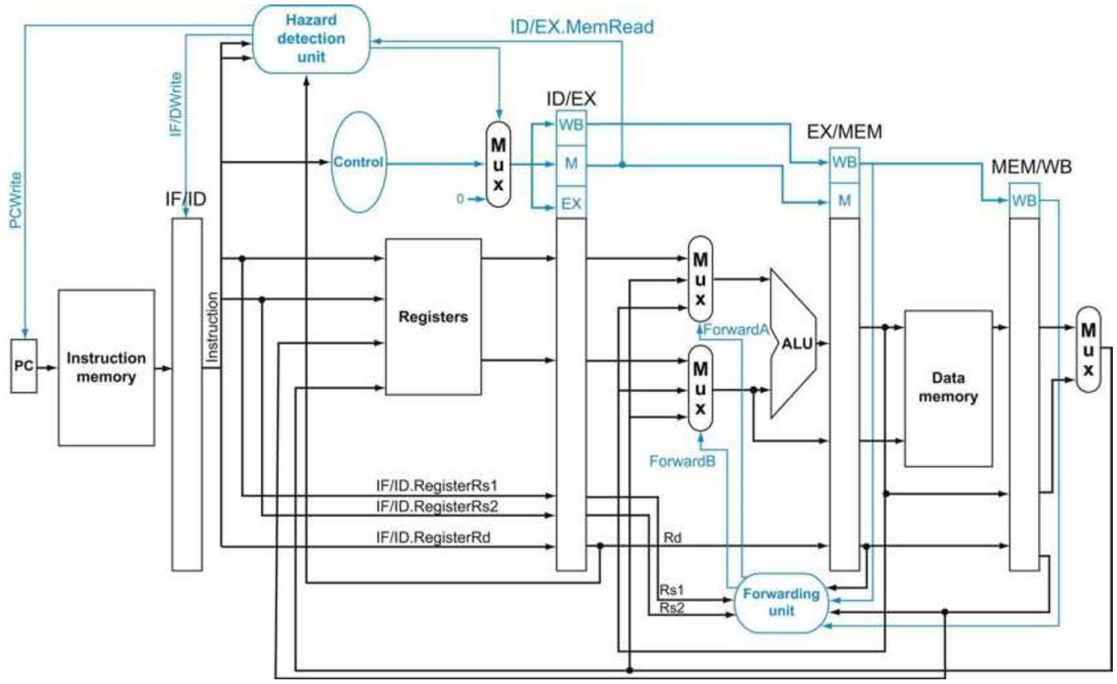
Git is a distributed version control system that helps developers manage and track changes in their source code efficiently. With Git, users can collaborate on projects, revert to previous code states, and maintain multiple development branches, making teamwork and code management more organized and reliable. Git is widely used in software development due to its speed, flexibility, and robust support for non-linear workflows. Please refer to this page for guidance on installing and configuring Git.

Report Requirements:

Your report should describe how you use Git to manage your source code. Include details about which files you track and which files you exclude using .gitignore. Additionally, explain how you use Git to collaborate effectively with your teammates. **[10 points]**

Appendix

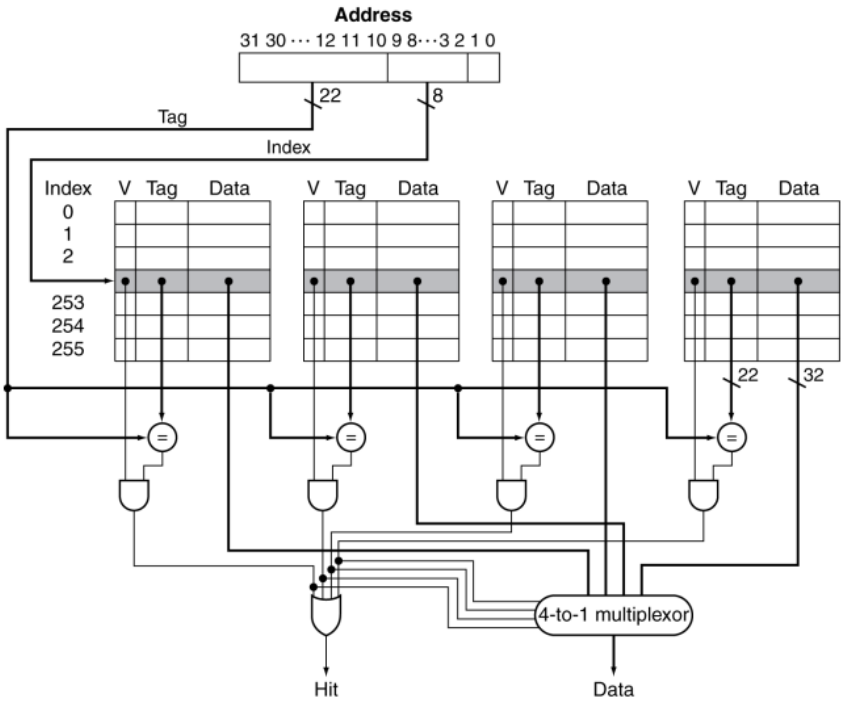
Pipelined RISC-V CPU



The **pipelined RISC-V CPU** is derived from the single-cycle RISC-V core and implements the RV32I ISA while splitting instruction execution across five pipeline stages to enable higher instruction throughput. The five stages are: Instruction Fetch (**IF**), Instruction Decode / Register Fetch (**ID**), Execute (**EX**), Memory Access (**MEM**), and Write Back (**WB**).

Your pipelined design must preserve the functional behavior of the single-cycle core while correctly handling pipeline hazards. Implement robust data-hazard handling (e.g., hazard detection, data forwarding/bypassing, and selective stalling or NOP insertion) so that Read-After-Write (RAW) and load-use dependencies are resolved without incorrect results. For control hazards, provide mechanisms such as branch prediction and pipeline flushing (and optionally early branch-target acquisition) to minimize misprediction penalties and ensure correct control flow. Where appropriate, apply ARM-like techniques (early BTA, careful use of clock edges, etc.) to reduce stalls and improve performance, but always preserve ISA semantics.

A 4-way set associative cache



A 4-way set associative cache can be inserted between memory and the CPU. The schematic of the 4-way set associative cache is shown above. The cache size is 4KB (256 rows × 4 ways × 4 bytes). The cache uses write-allocate and write-back scheme (D bit is not shown in the above figure). Integrating this cache will further add complexity to the Store and Load instructions. There are 4 situations when accessing the cache:

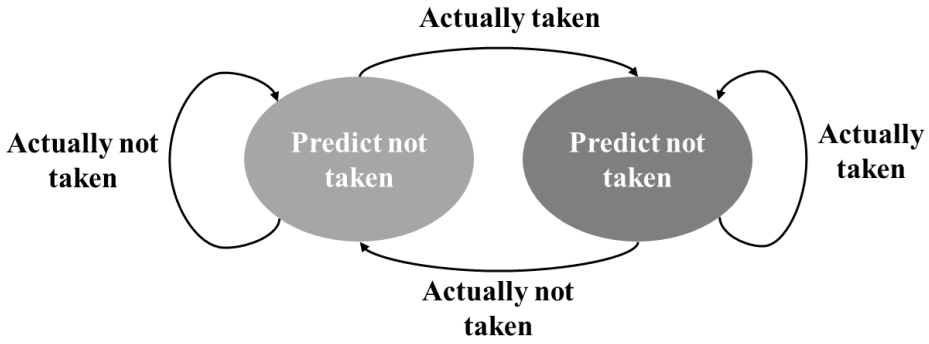
(1) When read hit, directly load data from cache to register.

(2) When read miss, load data from memory to cache, then load data from cache to register.

(3) When write hit, write to cache only, but set the block “dirty”, write back to memory when dirty data is replaced (write-back).

(4) When write miss, if the block is dirty, write the dirty block to memory first, then update the Tag/D bits and write the new data to the cache; if not dirty, write the new data to both cache and memory.

Dynamic Branch Prediction



Dynamic Branch Prediction is a technique used in modern CPU design to improve instruction pipeline efficiency when encountering branch instructions, such as conditional jumps (if, for, etc.). Instead of always making the same assumption about the outcome, the processor dynamically predicts whether a branch will be taken or not, based on the runtime behavior of the program.

The CPU maintains a history of previous branch results in a small memory called the branch history table. By analyzing past behavior, the processor becomes more accurate in its predictions. If the prediction is correct, the pipeline continues smoothly; if not, some instructions may need to be discarded and re-executed.